Supplementary Material for "Historical Confidentiality for Encrypted Agent Memory: Dual-Epoch Security and the Channels That Defeat It"

Shreyas Fegade

This document contains the full proof of Theorem 1, whose statement, conditions and proof sketch appear in the main paper. Numbering of theorems, definitions, propositions and sections follows the main paper throughout.

1 Proof of Theorem 1

Let Π satisfy S1 to S4, let F be the pseudorandom function used to derive record tags and the per-flush random choices, and let AE be the authenticated encryption scheme. Let $\mathcal{A}$ be a probabilistic polynomial-time adversary in $\mathbf{G}^{\mathrm{H}}$ that submits two equal-length histories — admissible or not — and issues polynomially many post-τ operations. The proof proceeds through three games and then exhibits the simulator of Definition 1.

Cryptographic model: AE is XChaCha20-Poly1305, used as an authenticated cipher that is IND$-CPA (ciphertexts are indistinguishable from uniform strings of the same length) and INT-CTXT. Nonces are 192-bit and drawn uniformly at random per encryption; the width makes a collision negligible ($q^2/2^{193}$ over q encryptions) and removes any nonce counter from disk. A header's associated data is a label naming the engine and format version together with the header's position (region and slot in Mnemos-P, absolute slot in Mnemos-M), so a header moved to another slot fails authentication; a payload's associated data is its record's identifier tag, which binds it to its header. Headers, payloads, the manifest, the table and the journal are sealed under distinct derived keys. The manifest and the partition table are themselves AE ciphertexts of fixed length under their own derived keys; the partition table is a ciphertext, not a PRF image, and is included in the snapshot as one of the fixed-length uniform strings of game G_2. All working keys are derived from the master by HKDF, treated as a PRF with independent labels per key, so the derived keys are independent. The simulator is given the public structural parameters $(B, C, d, \text{block size}, w, \Delta, k)$ and the leakage $\mathcal{L}$; it is *not* given b, the histories, or any plaintext. Adaptivity is permitted on physical locations the adversary has already observed, which the next paragraph treats.

On adaptivity: $\mathcal{A}$ chooses each post-τ operation after seeing the leakage of every earlier one. This does not disturb the argument. Each hybrid step below replaces a primitive with an idealized version, uniformly over the whole game, rather than fixing the operation sequence in advance; the reduction answers $\mathcal{A}$'s queries as they arrive and its advantage is unchanged. The final step shows the view is a function of the leakage and public randomness alone, which holds pointwise for every sequence $\mathcal{A}$ might produce and therefore holds for an adaptively chosen one.

Game G_0: The real game $\mathbf{G}^{\mathrm{H}}_{\Pi,\mathcal{A}}$.

Game G_1: As G_0, except that every evaluation of F under a store key is replaced by a lazily sampled random function. This affects the record tags, the keyed logical-to-physical partition permutation, the free-slot choice within a rewritten region, the choice of uniformly random padding regions in a flush, and the per-call probe randomization. A distinguisher between G_0 and G_1 yields a distinguisher against F, so $|\Pr[\mathrm{G}_0 \Rightarrow 1] - \Pr[\mathrm{G}_1 \Rightarrow 1]| \leq \mathsf{Adv}^{\mathrm{prf}}_F(\lambda)$ up to a factor for the number of derived keys.

Game G_2: As G_1, except that every ciphertext written to disk is replaced by a uniformly random string of the same length. This includes each slot's header and payload ciphertexts, the manifest, and the partition table, and every re-encryption performed when a region is rewritten during a flush. The challenger keeps the plaintexts of live records in its own state, so no stored ciphertext is decrypted. Each ciphertext is produced under a fresh uniformly random nonce and the number of encryptions is polynomial, so a reduction answering every encryption with an IND$-CPA challenge oracle gives $|\Pr[\mathrm{G}_1 \Rightarrow 1] - \Pr[\mathrm{G}_2 \Rightarrow 1]| \leq \mathsf{Adv}^{\mathrm{ind\$}}_{\mathsf{AE}}(\lambda)$.

Independence of the view in G_2 from b: The view consists of the snapshot at τ and the leakage of each post-τ operation. Each component is considered in turn.

1) *File lengths.* By S1 every file has a length determined

• *S. Fegade is an independent researcher.*
E-mail: shreyasf@icloud.com

by the public parameters, so the snapshot's shape is independent of b.

2) *Snapshot bytes.* By S2 and the change made in G_2, every slot of the partition file, whether it holds a live record or is free, is a uniformly random string of the slot length, and the manifest and table are uniformly random strings of fixed length. The snapshot is therefore a tuple of independent uniform strings whose count and lengths are public parameters, identically distributed under $b = 0$ and $b = 1$.
3) *Flush leakage.* By S3 a flush writes exactly $w+1$ whole regions. The regions written are: one per staged record, being that record's physical region; uniformly random regions to reach w; and one region fixed by a counter that depends only on the flush index. A staged record's physical region is the relabeling of the nearest-centroid partition of its embedding, a function of the embedding, the partition table and the relabeling, all independent of b by S4 and G_1. The uniform and counter choices are independent of b. The written bytes are uniform by item 2. So a flush reveals only a multiset of region indices with a distribution independent of b.
4) *Deletion and update leakage.* By S3 a deletion writes nothing at the time of the call and records a tombstone in the manifest, which is rewritten wholesale every flush regardless of content; an update is a deletion followed by a fresh write. The physical removal of a tombstoned record occurs only when the counter-selected region of a later flush is rewritten, which is already covered by item 3 and whose region choice is content-independent. Hence deleting or updating a cohort-H record adds nothing to the view that depends on which region held it.
5) *Search leakage.* A search reads the regions its probe set names. The probe set is a function of the query, the partition table, the relabeling and fresh per-call randomness, none of which depends on b. Ranking and payload decryption occur in memory over the region images already read, so no further disk access is issued.
6) *Refusals.* No admissibility assumption is needed: neither history is required to stay below capacity, and the case is closed by S3's padded refusal. Consider a post-τ put whose target partition is full under one history and not under the other. Under both, S3 requires the store to rewrite the intended region: an accepted write re-seals the region with the new record among its slots, and a refused write re-seals the same region unchanged. By item 2 every slot of a rewritten region is a uniform string of the slot length whether or not it holds a live record, so the two rewrites are identically distributed, and the region index written is the same function of the (relabeled) target partition in both cases. The observation extends to a put that overflows a partition *within* σ_b, before τ: the refused writes rewrite their regions, and by item 2 the snapshot does not reveal a region's occupancy, so a history that overflows a partition and one that does not produce identically distributed snapshots. Hence the refusal rule is exercised freely under both histories without distinguishing them, and no admissibility restriction is needed. What overflow costs is availability, not confidentiality: a padded refusal discards the caller's write. Proposition 5 states this as the capacity–confidentiality trade-off — padded refusal (chosen here) hides the load at the price of lost writes, silent refusal keeps the writes at the price of the load — and Section 7.7 measures both, including the utilization at which the losses begin.

Every component is thus a function of the public parameters, the leakage of the operations issued, and randomness independent of b. A simulator given only $\mathcal{L}$ reproduces the view: it samples the relabeling π itself, emits fixed-length uniform strings for the snapshot, and for each operation emits the region indices $\mathcal{L}^{\mathrm{op}}$ names under π, drawing the uniform padding and per-call probe randomness itself. Hence the real and simulated views are indistinguishable and $\Pr[b' = b] = \frac{1}{2}$ in G_2, giving

$$\mathsf{Adv}^{\mathrm{H}}_{\Pi,\mathcal{A}}(\lambda) \le \mathsf{Adv}^{\mathrm{prf}}_{F}(\lambda) + \mathsf{Adv}^{\mathrm{ind\$}}_{\mathsf{AE}}(\lambda) \le \mathsf{negl}(\lambda).$$

■

Predicate hiding (Corollary 1): Let $\mathcal{D}$ be a distribution over cohort-H histories, let ϕ be a predicate on cohort-H contents, and let $\mathcal{P}$ be the best predictor of ϕ that does not see the adversary's view. Suppose an adversary $\mathcal{B}$ predicts ϕ from its view with advantage δ over $\mathcal{P}$, where the advantage is taken in expectation over $\mathcal{D}$.

Construct a distinguisher $\mathcal{A}$ for $\mathbf{G}^{\mathrm{H}}$ as follows. $\mathcal{A}$ samples $\sigma_0 \leftarrow \mathcal{D} \mid \phi = 0$ and $\sigma_1 \leftarrow \mathcal{D} \mid \phi = 1$, submits them, runs $\mathcal{B}$ on the resulting view, and outputs $\mathcal{B}$'s prediction of ϕ as its guess for b. Conditioned on the sampled pair, $\mathcal{A}$ is correct exactly when $\mathcal{B}$ predicts ϕ correctly, so $\Pr[b' = b] = \frac{1}{2} + \frac{1}{2}\,\delta_{\sigma_0,\sigma_1}$ where $\delta_{\sigma_0,\sigma_1}$ is $\mathcal{B}$'s advantage on that pair. Averaging over the draw of the pair gives $\mathsf{Adv}^{\mathrm{H}}_{\Pi,\mathcal{A}} \ge \delta/2$, hence $\delta \le 2\,\mathsf{Adv}^{\mathrm{H}}_{\Pi,\mathcal{A}}(\lambda)$. The averaging is necessary because $\mathcal{B}$'s advantage may be concentrated on a small set of histories; the bound holds for the average, not pointwise. This is the standard equivalence between semantic security and indistinguishability, instantiated for the cohort-H game.

The direction of the corollary is worth restating, because it is easy to invert by accident. It converts a *proof* of dual-epoch security into a bound on predicate recovery. It does not convert a *measurement* into such a bound: an experiment exhibits one attacker, and the advantage it achieves is a lower bound on the leakage present, never an upper bound on the leakage available.

Necessity of the conditions: Condition S2 requires pseudorandom rather than merely CPA-secure ciphertexts because free slots are uniform strings that must be indistinguishable from live ones. Condition S3's whole-region rewrite is used in item 3: a per-slot write would reveal which slot changed and, accumulated, the occupancy that item 2 hides. S3's padded refusal is used in item 6 and is what lets the theorem cover overflow without an admissibility assumption; dropping it for silent refusal reopens the capacity channel of Proposition 5, which Section 7.7 measures. The flush at τ required by the game closes the staged-remainder channel. Propositions 1 and 4 show that dropping S1, S2 or S3 admits a distinguisher, which Section 7.4 realizes as ablations,

together with the observation that the keyed relabeling is *not* among the conditions and that removing it leaves the game at chance.

2 PROOF OF THEOREM 2

We prove that a store satisfying S1, S2 and the clock conditions C1–C3 is reactively $\mathcal{L}_{\mathrm{clock}}$-dual-epoch secure (Definition 2) against every polynomial-time feedback agent P. The structure mirrors the proof of Theorem 1; what changes is that the operation sequence after τ is produced by P reacting to plaintext search results, so the argument must show that this reaction never reaches the adversary's view.

Setup: Fix P and an adversary $\mathcal{A}$. In $\mathbf{G}^{\mathrm{R}}_{\Pi,\mathsf{P},\mathcal{A}}$ the challenger builds cohort H from σ_b, flushes at τ, and then for each adversary-supplied input runs one turn of P: a sequence of search operations whose plaintext results P reads, followed by put, delete or update operations that may depend on those results. The adversary receives $\mathcal{L}^{\mathrm{op}}$ after each turn and, separately, the clock leakage $\mathcal{L}_{\mathrm{clock}}$ of unlock, lock and flush times.

Game H_0: The real reactive game.

Game H_1: Replace every pseudorandom function under a store key by a lazily sampled random function, as in G_1. The gap is $\mathsf{Adv}^{\mathrm{prf}}_F(\lambda)$ up to the number of derived keys. This step is oblivious to who chose the operations, so P's reactivity does not affect it.

Game H_2: Replace every ciphertext written to disk by a uniformly random string of the same length (C2), keeping plaintexts in challenger state so no stored ciphertext is decrypted. The gap is $\mathsf{Adv}^{\mathrm{ind\$}}_{\mathsf{AE}}(\lambda)$. After this step every byte on disk, in every slot and in the sealed journal that carries staged records between flushes, is a uniform string whose length is a public parameter.

The view is a function of the clock: We show the adversary's view in H_2 is determined by $\mathcal{L}_{\mathrm{clock}}$ and public randomness, independently of b, of the inputs, and of every branch P took.

1) *Snapshot.* By S1 and H_2 the snapshot is a tuple of fixed-length uniform strings whose count and lengths are public, identical under $b = 0$ and $b = 1$.
2) *Writes.* By C1 the set of disk locations a flush rewrites is the image of the flush index under the deterministic sweep: the κ slots after a cursor that advances by κ each flush from a start fixed at unlock. This set is a function of the flush index alone. It does not depend on how many records P staged, on their content, or on whether P wrote at all — a flush occurs every Δ regardless (C1). By H_2 the bytes written are uniform. So the write transcript is the clock's sequence of cursor ranges, filled with uniform bytes.
3) *Reads.* By C3 a search issues no disk access after the unlock scan. Therefore the search operations P performs, and the results it reads, produce *no* storage event. This is the crux: the only place P's secret-dependent behaviour could enter the transcript is through the operations it issues, and its reads issue nothing while its writes are placed by the clock, not by content. The branch P takes is thus invisible.
4) *The journal.* Staged records wait in the fixed-size sealed journal, which C1 rewrites in full every flush whether it holds staged records or none; by H_2 its bytes are uniform of fixed length. Its rewrites are on the clock and carry nothing.
5) *Deletions and updates.* A deletion writes nothing at call time (C1 admits only the clocked sweep as a writer); its slot is freed when the sweep reaches it, an event already accounted in item 2. An update is a deletion and a staged write, both covered.

Every component is a function of the unlock/lock/flush clock and uniform bytes. A simulator given $\mathcal{L}_{\mathrm{clock}}$ emits the fixed-length uniform snapshot, and at each flush time emits the cursor range that flush index determines, filled with uniform bytes; it runs neither P nor the challenge, and never sees b or the inputs. Hence the real and simulated views are indistinguishable and

$$\mathsf{Adv}^{\mathrm{R}}_{\Pi,\mathsf{P},\mathcal{A}}(\lambda) \le \mathsf{Adv}^{\mathrm{prf}}_F(\lambda) + \mathsf{Adv}^{\mathrm{ind\$}}_{\mathsf{AE}}(\lambda) \le \mathsf{negl}(\lambda).$$

■

The lock: C1 includes one flush at lock, always. A lock is a time the adversary chose (Definition 2), so its flush is a clock event like any other, and the cursor range it writes is the next one in the sweep. A store that flushed *more* than once at lock when more records were staged than its journal holds would make the number of lock-time flushes a function of how much P wrote, which violates C1; this is why Mnemos-M's back-pressure, which refuses a put to the caller rather than stage beyond the journal, is the configuration Corollary 2 covers without exception.

Part (i): Let Π be operation-oblivious and let P issue at most q turns. Define hybrids $\mathrm{T}_0, \ldots, \mathrm{T}_q$, where in T_j the operations of the first j turns are replaced by the empty sequence at the same times and the remaining turns run P as in the real game. Consecutive hybrids differ in the operations of one turn, and by obliviousness the adversary's view of that turn, given everything before it, is indistinguishable under the two; the later turns are a function of that view and of P's state, which the adversary never sees. So $|\Pr[\mathrm{T}_{j-1} \Rightarrow 1] - \Pr[\mathrm{T}_j \Rightarrow 1]| \le \mathsf{negl}(\lambda)$, and in T_q the view is the snapshot followed by an idle schedule, both independent of b by S1, S2 and the obliviousness of idle turns across histories. Summing, $\mathsf{Adv}^{\mathrm{R}} \le q \cdot \mathsf{negl}(\lambda)$.

Part (ii): Fix a store state and two operation sequences O, O', searches included, whose leakage is at statistical distance ε, and let T be the optimal test between the two leakage distributions, with advantage ε. Let $x^\star$ be a fact absent from that state, let σ_1 produce the state and σ_0 produce it with $x^\star$ added, both padded to equal length. P, on input x, searches for x and issues O if the top result is a near-duplicate of x and O' otherwise. $\mathcal{A}$ supplies $x^\star$ and outputs $b' = 0$ exactly when T says O. Under $b = 1$ no near-duplicate exists and P issues O'; under $b = 0$ it issues O except with the probability δ that the search misses the stored duplicate. Two further differences remain between the worlds: the leakage of P's search, and the difference between the two store states on which O and O' run. An adversary of Definition 1 can issue the same search and the same operations on either state itself, so by dual-epoch security both differences are indistinguishable. Hence $\Pr[b' = b] \ge \frac{1}{2} + \frac{\varepsilon}{2} - \delta - \mathsf{negl}(\lambda)$,

and $\mathsf{Adv}^{\mathrm{R}} \geq \varepsilon/2 - \delta - \mathsf{negl}(\lambda)$. Proposition 6 is the case where O is a write and O' is nothing.

Why Mnemos-P is excluded: The proof uses C1 and C3 exactly where Proposition 6's attack strikes. Mnemos-P flushes on demand, violating C1: a flush occurs *because* P staged a record, so the occurrence of a flush is the write-occurrence bit a deduplicating agent leaks. And Mnemos-P places by content, so even under a clocked flush the region a reactive write lands in would depend on the retrieved plaintext. The clock-only store closes both, which is why the composition holds for it and not for the partitioned design — the separation $\mathrm{RDE} \Rightarrow \mathrm{DE}$, $\mathrm{DE} \not\Rightarrow \mathrm{RDE}$ of Corollary 2. Section 7.5 measures both sides.

3 Proofs of Lemma 2 and the Separations

Lemma 2: The placement rule is invariant under any permutation of the slot labels of r, and by hypothesis nothing else distinguishes them, so the labels are exchangeable. Hence the set $O_b(r)$ of slots cohort H occupies at τ is a uniformly random subset of size $L_b(r)$, whatever the history. Post-τ records draw uniformly without replacement from its complement, so k of them occupy a uniformly random ordered k-subset of the complement of a uniformly random $L_b(r)$-subset, which is a uniformly random ordered k-subset of all C labels whenever $k \leq C - L_b(r)$. The hypothesis that no partition is overfilled guarantees that bound under both histories, so the distribution does not depend on b. By S2 the snapshot does not reveal $O_b(r)$, so the adversary cannot condition on it.

Proposition 1: Take any forward- and Type-II-backward-private dynamic SSE scheme and adjoin to its client state a per-partition update counter, incremented on each put and stored under a pseudorandom handle in the snapshot. Forward and backward privacy are transcript notions and are unaffected by client-side state, so the augmented scheme keeps them. The snapshot now reveals a *multiset* of counter values: m records written to one partition yield $\{m\}$, $m/2$ written to each of two yield $\{m/2, m/2\}$, and no relabeling of partitions maps one to the other. The distinguisher submits those two histories and reads the multiset off the snapshot, so $\mathsf{Adv}^{\mathrm{H}} = 1/2$.

Proposition 2: Breach resistance asks that a single snapshot be simulatable from $\mathcal{L}^{\mathrm{snap}}$. That is the τ-only restriction of Definition 1, so a dual-epoch simulator stopped at τ is a breach-resistance simulator. For the converse, the witness of Proposition 1 is breach-resistant with respect to a snapshot leakage that includes the counter multiset, and forward/backward private, and is not dual-epoch secure with respect to the same leakage once the post-τ writes are read jointly with it: induced puts tell the adversary which counter is which topic, and the multiset then gives cohort H's per-topic load. Neither of the two notions constrains that joint object.

Proposition 3: History independence fixes the distribution of the stored state given its *contents*, so two insertion orders of one set are indistinguishable. The two challenge histories of Definition 1 may hold different sets. A content-placed, strongly history-independent store still holds a topic's records in that topic's region, and a growing layout reveals each region's size; with σ_0 placing m records on one topic and σ_1 spreading them over two, the snapshot's region sizes differ, and the distinguisher wins with advantage $1/2$. Proposition 4 isolates the two properties that remove the channel.

Proposition 4: With growing regions, a region's size is its occupancy, so the histories above are distinguished from one snapshot even though no order inside a region is revealed. With fixed-size regions whose free slots are not sealed, free slots are distinguishable from ciphertext (for instance, zero bytes), so the boundary between written and unwritten slots again gives each region's occupancy. Section 7.4 realizes both stores as ablations.

4 The Counting Study in Full

Proposition 1 separates forward and backward privacy from dual-epoch security by exhibiting state a forward-private scheme persists locally: a per-keyword update counter, which distinguishes two histories with different count *profiles*. The main paper reports the headline, that on a realistic corpus the channel is weak. This section gives the method behind that number.

Setup: The corpus is D3, whose keyword tags follow a heavy-tailed distribution with many ties, which is the property the attack turns on. Each record carries three tags drawn from a 503-term vocabulary. A Sophos-style index is built over it, and the snapshot exposes, for each keyword, a pseudorandom handle and the number of updates recorded under it. The adversary never sees the keyword itself.

The adversary: It holds auxiliary knowledge of keyword frequencies: a fraction α of the corpus in the clear, from which it estimates an expected count for every keyword. It then matches observed counters to expected ones, greedily and in descending order of expected count, which is the strongest simple matching available to it. The sweep runs $\alpha \in \{0.10, 0.25, 0.50\}$ against a range of tag skews.

Why it is weak here: Three effects compound. Most keywords are rare, so most counters are small integers. Small integers collide: the number of keywords sharing a given count grows sharply as the count falls, and keywords with equal counts are information-theoretically indistinguishable from counts alone. And the record-level question the paper actually cares about, the topic of a stored record, requires recovering several of a record's tags rather than one, so per-keyword errors compound rather than average out.

The *tie ceiling*, the fraction of keywords whose count is unique, bounds the recovery rate of any matching based on counts alone. Averaged over the sweep, keyword recovery is 0.05 and record-level topic accuracy never exceeds the most-frequent-class baseline. The worst case remains real, which is what makes the proposition a separation rather than an observation, but a deployment whose client state is sealed and padded to a fixed size removes the channel outright.

5 Query Equality in Full

Section 7.8 reports that neither probe mechanism hides query equality from an adversary that thresholds read-set overlap. This section gives the measurement in full (Table 1), the reason cover does not help, and the anonymity-set analysis, which is the property that separates the two mechanisms.

TABLE 1
Query equality against three adversaries ($B = 64$, $k_{\text{probe}} = 4$), over 20 independent stores with 16 queries each, every query searched 10 times in random order. The overlap threshold τ and the clustering threshold are fixed on pairs among half of each store's queries and scored on the other half, so no adversary is fitted to the pairs it is scored on. J is Youden's (TPR − FPR); ARI is the adjusted Rand index of the recovered query groups. Applies to the RW class only.

Probe mechanism	Regions/search	Exact-equality J	Threshold J	τ (mode)	Clustering ARI
Genuine probes only ($s = 1$)	4	0.99 [0.99, 1.00]	0.98 [0.97, 1.00]	4	0.94 [0.89, 0.99]
Randomised superset ($s = 2$, shipped)	8	0.00 [0.00, 0.00]	0.98 [0.97, 0.99]	4	0.66 [0.59, 0.71]
Randomised superset ($s = 4$)	16	0.00 [0.00, 0.00]	0.59 [0.57, 0.60]	6	0.13 [0.11, 0.15]
Grouped probes (group size 8)	8	0.85 [0.82, 0.88]	0.85 [0.82, 0.88]	1	0.59 [0.53, 0.65]

The adversary: It observes, for each search, the set of regions read, and must decide whether two searches carried the same query. It never sees a query or a result. The weak test is exact equality of the two read sets; the strong one compares the size of their intersection, $|R_1 \cap R_2|$, against a threshold τ; the third clusters the graph that joins searches whose overlap reaches τ. Each threshold is fixed on half of a store's queries and scored on the other half, over 20 independent stores, so no adversary is credited with a threshold fitted to the answers.

The randomized superset: A search reads $s\,k$ regions ($k = k_{\text{probe}}$), of which k are the genuine nearest partitions of the query and the remaining $(s-1)k$ are drawn with fresh randomness on every call. Two searches on the same query therefore almost never produce identical read sets, and the exact-equality test scores $J = 0.00$ at $s = 2$, which is what makes it look as though this mechanism closes the channel.

It does not, because the genuine probes survive every repeat. Treating cover as uniform, two searches for the same query share the k genuine regions plus about $(s-1)^2k^2/B$ cover collisions, and two unrelated queries share about s^2k^2/B, so the expected gap is

$$k\Big(1 - \frac{(2s-1)\,k}{B}\Big).$$

More cover narrows the gap linearly in s while the variance of both overlaps grows, and the gap closes only when $s\,k$ approaches $B/2$, where every search reads about half the store. At the shipped parameters the threshold test separates the populations with $J = 0.98$ $[0.97, 0.99]$; doubling the cover to $s = 4$ lowers it to $J = 0.59$ $[0.57, 0.60]$. We had registered the expectation that more cover would barely matter, with a lower bound of 0.60 on this interval, and it does matter: cover is a real knob. It is a costly one, though, since read volume grows in step with s and the channel stays open until a search reads half the store, at which point reading all of it, obliviously, costs little more.

The grouped mechanism: The partitions are divided into fixed disjoint groups at setup, and a search reads exactly one whole group. Two searches therefore read either the same regions or disjoint ones, so repeated-query recognition succeeds at the granularity of a group and $J = 0.85$ $[0.82, 0.88]$: every repeat is recognised, and two distinct queries are confused exactly when they fall in one group. What it buys in exchange is a guarantee the randomized superset cannot state: within a group every query is indistinguishable from every other, giving a deterministic anonymity set equal to the group size, independent of the adversary's effort. The randomized superset's protection is statistical and degrades with the number of observed repeats; the grouped mechanism's is structural and does not. Together with the permutation recovery of Section 7.8, which the grouped mechanism defeats outright, this is why it is the one preferred against a read observer despite recognizing repeats more often.

6 From Attacker Performance to Information Lower Bounds

The leakage ledger converts each channel's measured attacker performance into a lower bound, in bits, on the mutual information $I(X;V)$ between the secret X the channel is about and the adversary's view V. The conversion is Fano's inequality, and this section states it exactly for each metric family. In particular an AUC, which is a ranking statistic and not a decoding accuracy, is never fed to Fano as if it were one.

The inequality: For a secret X over K values and any decoder that recovers it from V with error probability P_e, Fano's inequality gives $H(X \mid V) \le h(P_e) + P_e \log_2(K-1)$, where h is the binary entropy. Hence

$$\begin{aligned} I(X;V) &= H(X) - H(X \mid V) \\ &\ge H(X) - h(P_e) - P_e \log_2(K-1). \end{aligned}$$

The ledger evaluates this at the *empirical* accuracy $1 - P_e$ of the attacker the experiment exhibited. It is a lower bound twice over: Fano bounds the information any decoder needs, and the experiment's decoder is one particular decoder, so a better one can only raise $I(X;V)$.

The prior term $H(X)$: The form $H(X) = \log_2 K$ is exact only for a *uniform* prior; under a skewed prior $H(X) < \log_2 K$ and that form can exceed the true $I(X;V)$, so it is not a valid lower bound. The ledger therefore records, per row, whether the prior is uniform *by construction* or empirical:

- *Binary games* (E5, the refusal channel E9, query equality E15, dedup E3): $K = 2$ and the secret is a fair challenge bit or a balanced held-out label, so $H(X) = 1$ exactly.
- *Permutation recovery* (E7): the secret is a region's logical partition under a *keyed* permutation, uniform over the B regions by construction, so $H(X) = \log_2 B$ exactly. This is why the headline "4.9 of 6 bits of partition identity" is a valid bound rather than an artifact of assuming uniformity.
- *Topic and region recovery* (E4, E6, E8): the prior is the corpus's topic (or region) distribution, which is skewed. The ledger uses the empirical entropy of that

distribution, recorded by the experiment; where a run predates this recording the row is flagged `uniform (assumed)` and its bound is the conservative $\log_2 K$ value, which for these rows overstates rather than understates and is marked as such.

Metric families, and why AUC is not converted:

- *Balanced accuracy* (binary games, dedup): used directly as $1 - P_e$, with the threshold fixed on a training half and the accuracy measured on a held-out half, so it is the accuracy the attacker *commits* to, not an in-sample optimum.
- *Youden's J* (refusal, query equality): for a single balanced pair the balanced accuracy of the thresholded test is exactly $(1+J)/2$, so J maps to accuracy without a ranking assumption.
- *Advantage over chance* (topic recovery): the accuracy is chance + advantage directly.
- *Precision* (multi-version): the probability the named region is correct is used as the decoding accuracy for a $K = B$ secret.
- *AUC* is **not** converted. AUC measures ranking, not decoding, and the earlier $1/4 + \mathrm{AUC}/2$ bound credited a perfect classifier with only 0.75 accuracy while crediting a useless one with more than chance — it is loose in one direction and unfounded in the other. Where a channel is measured only by AUC and no held-out accuracy was recorded, the ledger reports the AUC and *no* bit, rather than a fabricated one. Every headline bit in the paper comes from a row with a recorded accuracy or a J.

Units: The ledger states, per row, whether the secret is a binary history bit, a topic label, a region index, or a pair-equality bit, and $\log_2 K$ (or the empirical $H(X)$) is the ceiling against which the bits are read. A row's bound is never compared across units without that ceiling stated beside it.

7 Replication Units and Attacker Specification

Table 2 names, for each experiment, the unit an interval resamples and how many independent units there are. An interval is only as wide as the unit is independent: resampling records within one store measures sampling noise in that store, not the variation between stores, so every row resamples the outermost unit that was drawn afresh. Means over at least 20 units carry a percentile bootstrap interval (10,000 resamples); means over fewer carry a Student-t interval, because a percentile bootstrap over three to five values has only a handful of distinct resample means and undercovers. AUC intervals resample the held-out units. Every threshold, orientation or classifier is fitted on a training half of the units and scored on the other half; no parameter is tuned on the units it is scored on.

In the deployed-store measurements (E6, E16, E32) one unit is a full store build under a stacked encrypted file system: 10 builds per store in E6, 20 PostgreSQL clusters for pgvector in E16, and 10 per capture granularity in E32. A null there is reported as "not detected" with its interval, not as a closed channel.

TABLE 2
The independent unit behind every interval. "Fresh" means the unit draws its own store, cohort split and attacker fit.

Exp.	Unit (fresh per unit)	n	Interval
E1	trial (store, split)	20	bootstrap
E2	induction sequence (split, order, fit)	20	bootstrap
E3	probe	80/class	bootstrap, AUC
E4	centroid seed (table, store)	5/gap	Student-t
E5	game instance	1,000/bit	bootstrap, AUC
E6	seed (store build)	10/store	Student-t
E7	store	20	bootstrap
E8	trial (store, versions)	20	bootstrap
E9	trial (store)	20	bootstrap
E15	store (query split)	20	bootstrap
E16	seed (PostgreSQL cluster)	20	bootstrap
E17	trial (store, versions)	20	bootstrap
E18	probe	30/class	bootstrap, AUC
E20	probe	400	bootstrap, AUC
E21	session	40	descriptive
E22	store	20/point	bootstrap
E23	grid point	1	descriptive
E32	seed (PostgreSQL cluster)	$10/g$	Student-t

The learned attacker: The dual-epoch game (E5) uses one learned distinguisher, fixed before any arm was run: scikit-learn's `HistGradientBoostingClassifier` with 200 iterations, maximum depth 3 and `random_state` 0, and no other hyperparameter set or searched. Its features are the per-instance trace summary the arm computes from the observer's view (per-region write counts and their spread, and for slot-granular arms the number of distinct slots touched and the mean, maximum and standard deviation of their indices). The instances of both bits are shuffled together and split in half; the classifier is fitted on one half and its AUC and balanced accuracy, at a threshold fixed on the training half, are reported on the other. Each arm also reports one named hand-built statistic, oriented on the training half, so that a weak learner cannot make an arm look closed without the hand-built statistic disagreeing.

The reactive game (E20) has a one-dimensional observation per probe, so it uses no classifier. It scores three named trace statistics (flushes per turn, region writes per turn, and writes landing in the probe's content region), each oriented on the training half of the probes, and reports the one whose held-out AUC departs furthest from one half. Taking the maximum of three on the held-out half biases the reported AUC away from chance, which is the conservative direction for an arm claimed to be at chance: the clock-only arm is at chance even under that selection.

8 Full Results

The main paper tabulates the rows its arguments rest on. The tables in this section and the next give every row, and are generated from the same result files by the same code, so the two cannot disagree. Table 3 is the leakage ledger with every channel in one unit, and Table 4 lists every prediction registered for this revision with its outcome; a failed prediction is kept and marked. Figure 1 and Table 5 give the attack on a timestamped stream at every induction budget (main paper, Section 7.1). Table 6 gives the attack on memories a local model actually wrote at every budget (main paper, Section 7.3); each of its intervals resamples

independent induction sequences, each drawing its own cohort split, induction order and attacker fit. The longitudinal study reports different advantages at nominally similar budgets because its budget counts induced records per topic while this one counts turns, each a burst of w_{real} records; in records per topic the two agree. Table 7 gives the cost of every design at every size measured (main paper, Section 7.9).

9 Further Tables and Figures

These results are argued from in the main paper, which carries their headline in prose or in the leakage ledger (Table 3); the full tables are here. Table 8 places the designs on the architectural frontier and Table 9 collects the notation. Figure 2 draws the τ-arrival adversary and Fig. 3 the cost measures against n. Table 10 is the drift study, discussed below. Table 11 gives the deployed vector stores under an encrypted file system, Table 12 deduplication membership inference, Table 13 recovery of the keyed permutation from search traffic, and Table 14 what a sequence of file versions reveals. Tables 15 and 16 give the refusal channel and capacity under a skewed trace, Table 17 retrieval quality on the full corpora, and Table 18 the cost of fetching payloads once reads are visible.

Two features of the curve are systematic. The advantage turns *negative* at the largest gap (-0.02, an interval below zero): the attacker commits to a partition-to-topic map fitted on the induced era, and once cluster geometry has drifted that map mislabels cohort H more often than a uniform guess. The gap-zero advantage (+0.42) exceeds the i.i.d. shuffled control (+0.37) because the shuffle mixes every era and dilutes the era-specific partition structure that same-era induction matches; the comparison that isolates drift is gap zero against the far eras. Cohort H on this corpus is skewed, its most frequent topic being 0.50 of its records, so Table 10 reports two further measures. Over an adversary that knows the prior and always guesses that topic, the fully archival adversary's raw accuracy is only level (0.004 [-0.026, 0.033]). Raw accuracy is the wrong yardstick here, because an adversary can make correct record-by-record guesses and still score below always naming the commonest topic. Balanced accuracy, the mean per-topic recall, is the right one: the prior alone scores exactly chance on it, and the archival adversary's advantage is 0.217 [0.151, 0.283], the current-content one's 0.055 [0.027, 0.082]. The labels therefore carry information about individual records beyond the topic mix, and drift erodes it. This measure was added after the runs, on the same induction sequences, and was not registered.

The operating-system artifact audit: Section 7.11 summarises it. On the evaluation machine the audit recovers change-journal (USN) records naming the store's own files, confirming that the file system retains pre-τ write history with timestamps; it enumerates volume shadow copies and reports where elevation was required to do so; it identifies which synchronization clients are installed and whether the store lies inside a synchronized tree; and it reports the hibernation file and pagefile, which matter because a hibernation image captures locked pages and therefore the unwrapped master key. It also verifies the mechanism a power-event handler would invoke: after locking, both sealed engines report themselves locked and refuse to search.

10 Agent-Memory Frameworks, Read from Source

Section 4.5 states that four of nine widely used open-source agent-memory frameworks make whether a write reaches storage depend on whether the content is already stored, which is the precondition of Proposition 6. Table 19 gives the evidence. The method is deliberately narrow: each framework's wheel was downloaded from PyPI without its dependencies and its shipped Python source read directly, and a mechanism is recorded only if it was located at the cited file and line. Documentation was not used, because documentation describes intent and the attack depends on behaviour.

The census also corrects a description of mem0 that is widely repeated. mem0 is commonly described as running an update stage that returns NOOP for a fact it already holds. The current release has no such stage: it passes the most similar existing memories to a single extraction call, and then drops any extracted fact whose hash is already present. The observable consequence is the same — a known fact never reaches the vector store — but it now arises from two mechanisms rather than one, and only the vector store's files are written conditionally: mem0's message log is written either way. An adversary that watches the whole store directory sees the difference as *which* files changed.

11 The Dual-Epoch Game: Per-Ablation Details

Section 7.4 reports the outcome of the game of Definition 1 against the compiled engines. This section gives the construction of the arms, and why four choices in the experiment are load-bearing rather than incidental. Each is a way the experiment can appear to measure the theorem's setting while measuring something else.

Admissibility is verified, not assumed: A history is admissible only if no partition exceeds its capacity at any point. An experiment that merely *intends* admissibility and wraps each write in a bare exception handler will silently submit inadmissible histories and measure the refusal channel, while reporting a number that reads as placement leakage. Every admissible arm here therefore asserts that the engine's own refusal counter is exactly zero at the end of the arm.

Two distinct defects were caught only by that assertion. The first was an admissibility check that recomputed each record's partition in floating point by nearest centroid, which disagrees with the engine's quantised assignment at the margin; the check now calls the engine's own assignment directly, so the experiment and the implementation cannot differ about where a record went. The second was subtler: bounding the pre-τ cohort at the partition capacity left no room for the adversary's own targeted post-τ writes, which then overflowed the very partitions the history had been constructed to keep admissible. The cohort bound is now set below capacity by the adversary's entire induced budget.

The second defect is the reason the assertion is worth its cost: at 50 instances per bit every admissible arm reported zero refusals and the experiment looked correct, and only at 1,000 instances per bit did an admissible arm report seven.

TABLE 3

Every channel in one unit. *Bits* is a Fano lower bound on the mutual information between the secret and what the adversary observes, per observation of the unit the channel is about (a record's topic, a region's partition, one game instance), taken at the lower end of the experiment's 95% interval so that it holds at the experiment's confidence; * marks a result with no interval, for which only the point bound exists, and † a channel measured only by AUC, which is a ranking statistic and carries no decoding-accuracy bound (the supplement derives the conversion). A test bounds leakage from below, so for a *not detected* channel the informative number is the last column: the smallest effect, in the experiment's own unit, it had 80% power to detect. Where cohort H's topic prior is skewed (the drift study and the agent-written streams), a leak is judged on balanced accuracy, the mean per-topic recall, which knowing the prior cannot raise; the Fano bound, computed from raw accuracy and the prior's empirical entropy, is then conservative and can be zero for a channel that leaks. Every measured channel, including every ablation.

Channel	Design	Class	Status	Bits ≥	Detectable ≥
structure transfer, historically aware adversary	growing layout	W	**leaks**	0.000	–
structure transfer, naive (current content) adversary	growing layout	W	**leaks**	0.000	–
structure transfer, agent-written memories (E1)	growing layout	W	**leaks**	0.000	–
structure transfer, agent-written memories (E2, bursts of 4)	growing layout	W	**leaks**	0.000	–
snapshot + post-τ writes (the game)	Mnemos-P	W	not det.	0.002	0.052 (AUC)
game, S1 removed (growing regions)	Mnemos-P ablation	W	**leaks**	1.000	–
game, S2 removed (unsealed slots)	Mnemos-P ablation	W	**leaks**	0.784	–
game, per-slot writes, random slot	Mnemos-P ablation	W	not det.	0.006	0.051 (AUC)
game, no keyed permutation	Mnemos-P ablation	W	not det.	0.000	0.052 (AUC)
refusal, padded	Mnemos-P	W	not det.	0.001	0.051 (AUC)
refusal, silent	Mnemos-P ablation	W	**leaks**	0.682	–
snapshot + post-τ writes (the game)	Mnemos-M	W	not det.	0.001	0.051 (AUC)
game, content placement	Mnemos-M ablation	W	not det.	0.008	0.051 (AUC)
game, first-free slot, whole-region writes	Mnemos-P ablation	W	not det.	0.000	0.052 (AUC)
game, per-slot writes, first-free slot	Mnemos-P ablation	W	**leaks**	1.000	–
structure transfer, deployed store: append_control	append_control	W	not det.	0.000	0.002 (advantage)
structure transfer, deployed store: chroma	chroma	W	not det.	0.004	0.020 (advantage)
structure transfer, deployed store: faiss	faiss	W	not det.	0.000	undetermined
structure transfer, deployed store: lancedb	lancedb	W	not det.	0.000	undetermined
structure transfer, deployed store: lancedb_indexed	lancedb_indexed	W	not det.	0.000	undetermined
structure transfer, deployed store: sqlitevec	sqlitevec	W	not det.	0.000	0.005 (advantage)
structure transfer, deployed store: pgvector	pgvector	W	**leaks**	0.048	–
permutation recovery (spread_s1)	Mnemos-P	RW	**leaks**	4.905	–
permutation recovery (spread_s2)	Mnemos-P	RW	**leaks**	1.827	–
permutation recovery (grouped)	Mnemos-P	RW	**leaks**	0.000	–
multi-version diff, g=1	Mnemos-P	W	**leaks**	0.524	–
multi-version diff, g=2	Mnemos-P	W	**leaks**	0.173	–
multi-version diff, g=4	Mnemos-P	W	**leaks**	0.049	–
multi-version diff, g=16	Mnemos-P	W	**leaks**	0.002	–
multi-version diff, g=64	Mnemos-P	W	**leaks**	0.000	–
multi-version diff, across a session, rewrite_at_lock off	Mnemos-P	W	**leaks**	0.031	–
multi-version diff, across a session, rewrite_at_lock on	Mnemos-P	W	not det.	0.000	undetermined
refusal, silent	Mnemos-P	W	**leaks**	0.763	–
refusal, padded	Mnemos-P	W	not det.	0.000	undetermined
query equality (spread_s1), overlap threshold	Mnemos-P	RW	**leaks**	0.887	–
query equality (spread_s2), overlap threshold	Mnemos-P	RW	**leaks**	0.890	–
query equality (spread_s4), overlap threshold	Mnemos-P	RW	**leaks**	0.252	–
query equality (grouped), overlap threshold	Mnemos-P	RW	**leaks**	0.566	–
dedup membership (threshold)	Mnemos-P	W	**leaks**	1.000	–
dedup membership (threshold)	Mnemos-M	W	not det.	0.000	undetermined
dedup membership (llm)	Mnemos-P	W	**leaks**	0.041	–
dedup membership (llm)	Mnemos-M	W	not det.	0.000	undetermined

A leakage measurement that is sound at low power and unsound at high power is indistinguishable from a sound one until the moment it matters.

Evaluation is held out: The distinguisher is fitted on one half of the instances and scored on the other. Without the split, a classifier with enough features recovers the bit on the data it was fitted to, and the resulting AUC measures capacity rather than leakage. With it, an AUC below one half is read as the small-sample anti-learning it is, and is not reported as evidence of a distinguisher running backwards.

Every arm carries a hand-built statistic: Alongside the gradient-boosted classifier each arm names one statistic, oriented on the training half, that a reader can check by hand: the spread of post-τ writes over regions for the shipped arms, and for the ¬S3 arm the slot-level features a per-slot write exposes — the number of distinct slots touched and the mean, maximum and standard deviation of their indices. A learned AUC with no interpretable companion cannot distinguish a real channel from a harness artifact; the pair can.

The inadmissible case is its own arm: Rather than being excluded, inadmissibility is run deliberately, in both refusal modes. This is what keeps the refusal channel from contaminating arms that claim to measure placement: the channel is given somewhere to appear, and Proposition 5 predicts exactly where. The padded refusal mode is the same arm with the fix applied, so the two together measure the countermeasure rather than merely asserting it.

TABLE 4
Every prediction registered for the later experiments, committed before its run, with the measured value. A failed prediction is kept; the verifier's range for it is widened only to the claim that survives, and its note records the original prediction and the numbers.

Check	Registered	Measured	Outcome
E16x pgvector adv_hist lower, 20 seeds	[0, 1]	0.094	held
E16x pgvector adv_hist, 20 seeds	[0.05, 0.3]	0.118	held
E32 pgvector adv_hist lower, g=1	[0, 1]	0.155	held
E32 pgvector adv_hist lower, g=4	[0, 1]	0.043	held
E32 pgvector adv_hist, g=16 minus g=1	[-1, 0]	-0.187	held
E29 mem0 on Qdrant: oracle AUC	[0.65, 1]	0.830	held
E29 mem0 on Mnemos-M, store observed: oracle AUC	[0.4, 0.6]	0.467	held
E29 mem0 on Mnemos-M, store observed: AUC upper	[0.5, 1]	0.562	held
E29 mem0 on Mnemos-M, store and history observed: oracle AUC	[0.65, 1]	0.629	**failed**
E29 mem0 on Mnemos-P, store observed: oracle AUC	[0.65, 1]	0.693	held
E30 LangMem oracle AUC	[0.6, 1]	0.500	**failed**
E30 LangMem oracle AUC, bytes appended	[0.6, 1]	0.462	**failed**
E30 LangMem non-member minus member write rate	[0.05, 1]	0.000	**failed**
E31 llama3.1:8b single-probe oracle AUC	[0.6, 1]	0.475	**failed**
E31 qwen2.5 AUC gain, sequential k=3 over one probe	[0, 1]	0.003	held
E31x additional models measured	[2, 2]	2.000	held
E31x models whose oracle matches their write-rate gap	[1, 1]	1.000	held
E33 agent-paced: flushes per retained Syncthing version	[0.9, 3]	3.050	**failed**
E33 agent-paced: version-diff precision over the 1/B null	[3, 64]	4.659	held
E33 agent-paced: exposure of a record's region	[0.9, 1]	1.000	held
E33 fast: flushes per retained Syncthing version	[5, 30]	13.880	held
E33 fast: version-diff precision over the 1/B null	[1, 2]	1.443	held

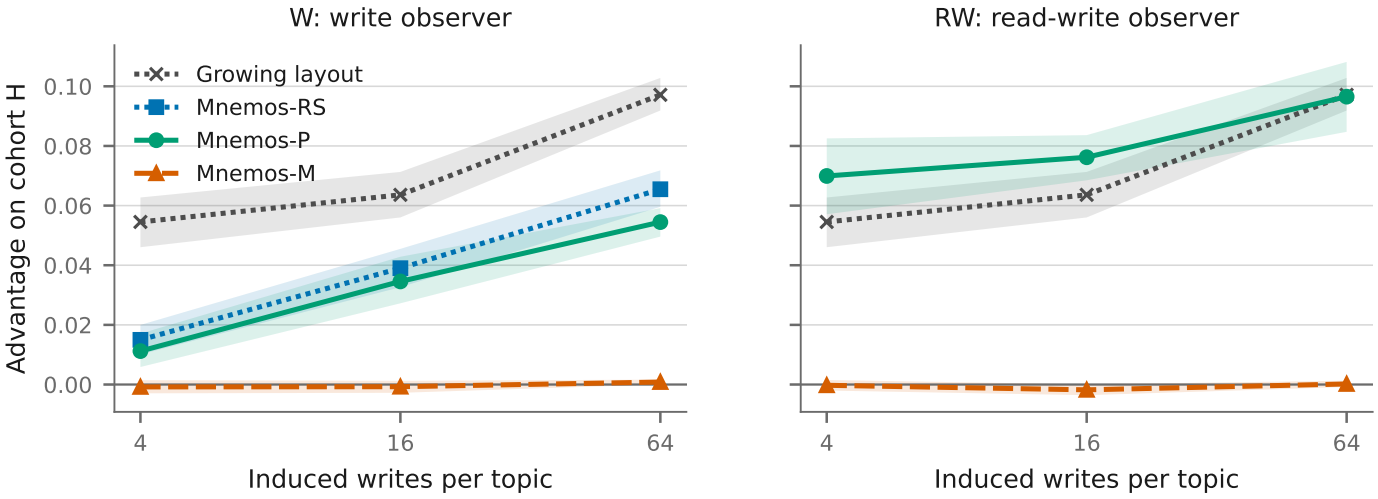


Fig. 1. Exposure of the pre-compromise cohort against the adversary's induction budget, on a timestamped stream compromised at a month boundary, for the write observer (left) and the read-write observer (right); bands are 95% intervals over twenty trials. The growing layout's snapshot discloses which region holds each earlier record. For the sealed designs the harness supplies that membership, so their curves are what the labels would be worth to an adversary that obtained it elsewhere, for instance from version diffs. Mnemos-RS is measured for the write observer, since its read-side channel is result-dependent reads. The supplementary material tabulates every point.

TABLE 5
Exposure of the pre-compromise cohort on a timestamped stream (2,501 records over 4 months, τ at month 2, 10 topics, chance 0.100). Source: agent-authored memories. *Membership* is how the attack learns which region holds a given earlier record: the growing layout's snapshot discloses it, while for the sealed designs the harness supplies it, so those rows state what the labels would be worth to an adversary that obtained membership elsewhere (Section 7.7). The RW adversary against Mnemos-P recovers the keyed permutation at the rate Section 7.8 measured, 0.564, and labels partitions from public text only.

Design	Class	Budget	Membership	$\mathrm{Adv}_{\mathrm{hist}}$ [95% CI]
Mnemos-M	W	4	supplied	-0.001 [-0.003, 0.001]
Mnemos-M	RW	4	supplied	-0.000 [-0.002, 0.002]
Mnemos-M	W	16	supplied	-0.001 [-0.003, 0.001]
Mnemos-M	RW	16	supplied	-0.002 [-0.004, 0.000]
Mnemos-M	W	64	supplied	0.001 [0.000, 0.002]
Mnemos-M	RW	64	supplied	0.000 [-0.001, 0.001]
Mnemos-P	W	4	supplied	0.011 [0.006, 0.017]
Mnemos-P	RW	4	supplied	0.070 [0.057, 0.083]
Mnemos-RS	W	4	supplied	0.015 [0.010, 0.020]
Mnemos-P	W	16	supplied	0.035 [0.027, 0.043]
Mnemos-P	RW	16	supplied	0.076 [0.068, 0.084]
Mnemos-RS	W	16	supplied	0.039 [0.033, 0.045]
Mnemos-P	W	64	supplied	0.054 [0.050, 0.059]
Mnemos-P	RW	64	supplied	0.097 [0.085, 0.108]
Mnemos-RS	W	64	supplied	0.065 [0.060, 0.072]
Growing layout	W	4	snapshot	0.055 [0.046, 0.063]
Growing layout	RW	4	snapshot	0.055 [0.046, 0.063]
Growing layout	W	16	snapshot	0.064 [0.056, 0.071]
Growing layout	RW	16	snapshot	0.064 [0.056, 0.071]
Growing layout	W	64	snapshot	0.097 [0.092, 0.103]
Growing layout	RW	64	snapshot	0.097 [0.092, 0.103]

TABLE 6
Every budget measured. The attack on 3,666 memories a local model actually wrote (10 topics, chance 0.100). w_{real} is the burst size: how many same-topic memories one writing turn contributes to a flush. The growing layout discloses each record's partition in the snapshot and is the positive control; the advantage is scored for an adversary that guesses where it has no label.

Layout	w_{real}	Turns/topic	$\mathrm{Adv}_{\mathrm{hist}}$ [95% CI]
growing (control)	1	4	0.041 [0.024, 0.058]
growing (control)	1	8	0.037 [0.016, 0.062]
growing (control)	1	16	0.027 [0.009, 0.047]
growing (control)	1	32	0.051 [0.017, 0.091]
growing (control)	2	4	0.109 [0.085, 0.130]
growing (control)	2	8	0.076 [0.057, 0.094]
growing (control)	2	16	0.149 [0.113, 0.179]
growing (control)	2	32	0.158 [0.119, 0.191]
growing (control)	4	4	0.085 [0.076, 0.094]
growing (control)	4	8	0.182 [0.172, 0.193]
growing (control)	4	16	0.184 [0.172, 0.196]
growing (control)	4	32	0.188 [0.173, 0.203]
Mnemos-P	1	4	0.001 [-0.003, 0.005]
Mnemos-P	1	8	0.001 [-0.002, 0.005]
Mnemos-P	1	16	0.009 [0.001, 0.017]
Mnemos-P	1	32	0.006 [0.000, 0.011]
Mnemos-P	2	4	0.015 [0.009, 0.021]
Mnemos-P	2	8	0.020 [0.013, 0.028]
Mnemos-P	2	16	0.031 [0.021, 0.040]
Mnemos-P	2	32	0.015 [0.010, 0.020]
Mnemos-P	4	4	0.040 [0.034, 0.046]
Mnemos-P	4	8	0.062 [0.051, 0.073]
Mnemos-P	4	16	0.036 [0.026, 0.048]
Mnemos-P	4	32	0.023 [0.017, 0.030]

TABLE 7
Cost at every size measured. Against the in-tree Rust plaintext IVF, which shares the quantisation and the partitioner, so a ratio is a statement about the security machinery. *Put* is the sparse regime: one record, one flush, which is what an agent writing seconds apart pays. *kB written/record* is what a whole-region rewrite costs per record; a dash marks a design that does not count its bytes or keeps nothing on disk.

Design	n	Search $p50$ (ms)	Put $p50$ (ms)	kB written/record	RAM (MB)	Disk (MB)	Unlock (s)
Plaintext IVF (Rust)	1,000	0.091	0.04	–	0.0	0.0	0.00
Mnemos-M, exact	1,000	0.137	4.33	198	5.5	5.8	0.21
Mnemos-M, in-memory IVF	1,000	0.068	3.81	198	5.5	5.8	0.20
Mnemos-P	1,000	1.237	15.00	198	0.0	5.8	0.25
Mnemos-RS	1,000	0.662	3.70	–	0.0	5.8	0.30
Plaintext IVF (Rust)	3,000	0.069	0.02	–	0.0	0.0	0.00
Mnemos-M, exact	3,000	0.454	6.04	290	16.4	17.2	0.21
Mnemos-M, in-memory IVF	3,000	0.104	5.84	290	16.4	17.2	0.23
Mnemos-P	3,000	1.718	17.24	290	0.0	17.2	0.23
Mnemos-RS	3,000	0.823	3.92	–	0.0	17.2	0.32
Plaintext IVF (Rust)	10,000	0.106	0.02	–	0.0	0.0	0.00
Mnemos-M, exact	10,000	1.467	8.96	478	54.5	57.1	0.26
Mnemos-M, in-memory IVF	10,000	0.238	8.12	478	54.6	57.1	0.38
Mnemos-P	10,000	2.557	22.15	478	0.0	57.1	0.28
Mnemos-RS	10,000	1.522	5.65	–	0.0	57.2	0.36
Plaintext IVF (Rust)	30,000	0.288	0.05	–	0.0	0.0	0.00
Mnemos-M, exact	30,000	6.404	13.13	713	163.3	170.8	0.35
Mnemos-M, in-memory IVF	30,000	0.602	14.04	713	163.5	170.8	1.21
Mnemos-P	30,000	3.813	28.19	713	0.0	170.8	0.39
Mnemos-RS	30,000	2.383	8.38	–	0.0	171.1	0.43
Plaintext IVF (Rust)	100,000	0.422	0.06	–	0.0	0.0	0.00
Mnemos-M, exact	100,000	23.194	32.18	1,185	543.7	568.2	0.79
Mnemos-M, in-memory IVF	100,000	1.778	33.85	1,185	544.3	568.2	6.67
Mnemos-P	100,000	6.667	46.26	1,185	0.0	568.2	0.69
Mnemos-RS	100,000	3.442	13.09	–	0.0	569.4	0.71

TABLE 8
The architectural frontier. Security is stated by the strongest adversary class each design closes the historical channels against; performance is orders of magnitude, not a benchmark. Figures marked $*$ are reported by the cited systems for their own settings and are not re-measured here; unmarked figures for the Mnemos designs are from Section 7.9 of the main paper. The point is the boundary between designs, not a head-to-head: these systems assume different adversaries.

Design	Closes hist. vs	Insert	Search	Resident	Disk/op	>mem	Ingest
Plaintext IVF	—	$O(1)$	sublinear	index	$O(1)$	yes	yes
Mnemos-P	W (not deriv.)	flush	sublin., disk	index	region	yes	yes
Mnemos-M	W, derived	clock	$O(n)$, mem	idx+payload	sweep	no	yes
Mnemos-M/disk	W, derived	clock	sublin., disk	posmap+part	sweep	yes	yes
Write-only ORAM	W	$O(1)^*$	writes only	posmap*	$O(1)^*$	yes	yes
Path ORAM slots	W, RW	$O(\log^2 n)^*$	$O(\log^2 n)^*$	stash*	$O(\log^2 n)^*$	yes	yes
Compass	W, RW	$O(\log)^*$	oblivious*	client*	per-acc*	srv	yes*
Onyx	W, RW	—	oblivious*	sublin.*	per-acc*	yes*	—
Opal (TEE)	W, RW	$O(\log)^*$	oblivious*	enclave*	per-acc*	yes*	yes*

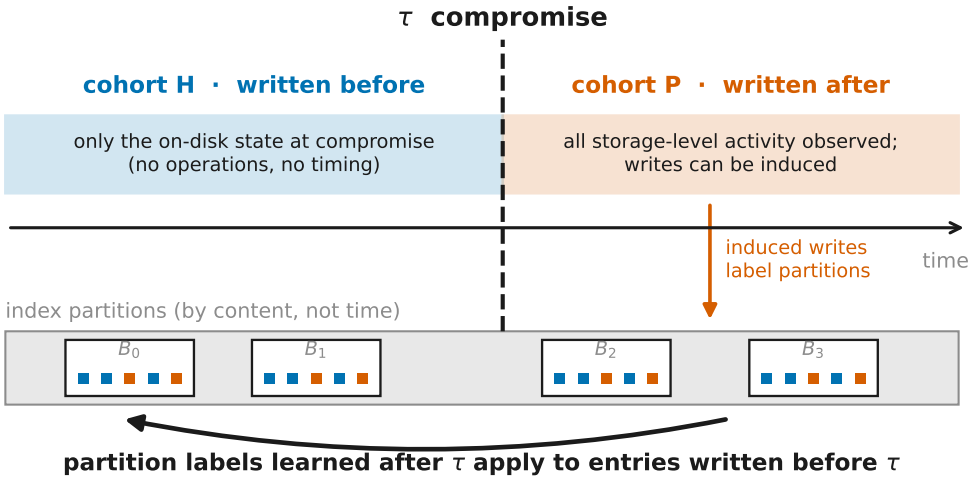


Fig. 2. The τ-arrival adversary. Records written before τ (cohort H) are available only as stored state; activity after τ (cohort P) is observed and partly induced. Where assignment depends on content, partitions hold records from both cohorts, so labels learned after τ describe cohort H as well — but they reach a given cohort-H record only if the layout discloses which partition holds it, which a growing layout does and a sealed fixed-size one does not (Lemma 1).

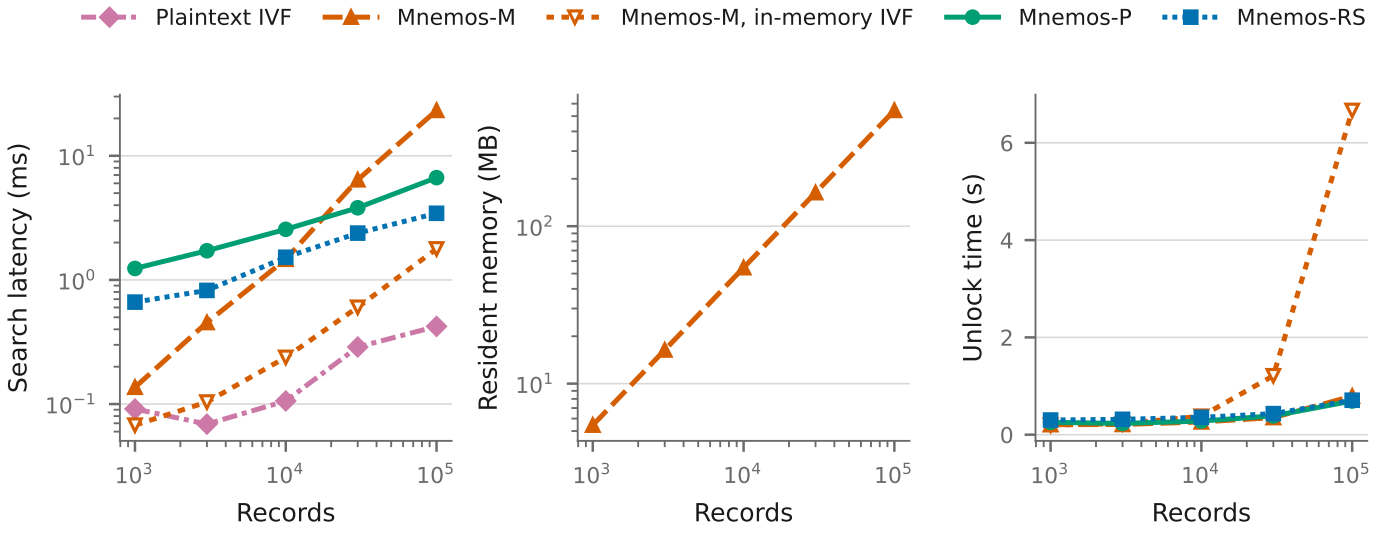


Fig. 3. Where the choice between the designs is decided (main paper, Section 7.9). Three measures on different scales are shown as small multiples over a shared n. Mnemos-M's exhaustive ranking is linear in n and crosses the partitioned design between 10,000 and 30,000 records; with its partition index held in memory (hollow markers) it stays ahead at every size, paid for at unlock, where every record's partition is recomputed. The middle panel, resident memory, is what bounds Mnemos-M; the index leaves it unchanged.

TABLE 9
Notation

Symbol	Meaning
B, C	partitions (or regions); slots per partition
w	records written per flush (*epoch_writes*)
Δ, k	clocked flush interval; slots per sweep flush
k_{probe}, top_k	partitions probed per search; records returned
s	probe spread
τ	time of compromise
H, P	records written before τ; at or after τ
β	injection budget: induced writes per topic
g	version-capture granularity, in flushes
$\mathrm{Adv}_{\mathrm{hist}}$, $\mathrm{Adv}_{\mathrm{post}}$	topic-recovery advantage on cohorts H and P

TABLE 10
Advantage on cohort H against the time gap between cohort H and the content the adversary induces (github, 12 topics, chance 0.083). Gap 0 is an adversary inducing with archival content from cohort H's own era; the largest gap is one inducing with current content. The i.i.d. control is 0.368 [0.351, 0.386]. Cohort H's topic prior is skewed: its most frequent topic is 0.504 of its records. *Over majority* is accuracy over an adversary that knows that prior and always guesses the most frequent topic. The last column is the advantage in balanced accuracy (mean per-topic recall), which knowing the prior cannot raise.

Gap (eras)	Adversary	$\mathrm{Adv}_{\mathrm{hist}}$ [95% CI]	Over majority [95% CI]	Balanced [95% CI]
0	historically aware	0.424 [0.394, 0.454]	0.004 [-0.026, 0.033]	0.217 [0.151, 0.283]
1	intermediate	0.246 [0.200, 0.291]	-0.174 [-0.220, -0.129]	0.426 [0.371, 0.480]
2	intermediate	0.036 [0.018, 0.054]	-0.384 [-0.402, -0.367]	0.343 [0.298, 0.388]
3	intermediate	-0.017 [-0.032, -0.003]	-0.437 [-0.452, -0.423]	0.225 [0.132, 0.318]
4	intermediate	-0.029 [-0.038, -0.020]	-0.449 [-0.458, -0.440]	0.124 [0.028, 0.220]
5	naive (current content)	-0.022 [-0.027, -0.017]	-0.442 [-0.447, -0.437]	0.055 [0.027, 0.082]

TABLE 11
Deployed vector stores under a `gocryptfs` mount, 10 repeats each, 8 topics (chance 0.125). Induced writes are *interleaved across topics*, so consecutive same-topic writes cannot make recency look like content placement, and block labels are fitted on one half of them and scored on the other. $\mathrm{Adv}_{\mathrm{hist}}$ is the quantity that matters: whether those labels identify the topics of records written *before* the compromise.

Store	Placed by	Blocks/write	Localisation, held out	$\mathrm{Adv}_{\mathrm{hist}}$
`append-control`	time	2.2	0.13 [0.11, 0.15]	0.00 [-0.00, 0.00]
`chroma`	content	12.5	0.14 [0.11, 0.16]	-0.01 [-0.02, 0.00]
`faiss`	content	367.7	0.12 [0.08, 0.17]	0.00 [0.00, 0.00]
`lancedb`	time	12.1	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
`lancedb-indexed`	content	243.9	0.00 [0.00, 0.00]	0.00 [0.00, 0.00]
`sqlitevec`	time	11.5	0.13 [0.11, 0.15]	-0.00 [-0.00, 0.00]

TABLE 12
Deduplication membership inference: whether an adversary learns that a fact was already in cohort H, from the occurrence of a write alone (80 probes per class, bootstrap intervals). The write rates are the agent's decisions and are the same for both designs; Mnemos-M's clocked flush makes what the adversary observes identical either way.

Decided by	Design	AUC [95% CI]	Writes, member	non-member
similarity	Mnemos-P	1.00 [1.00, 1.00]	0.00	1.00
similarity	Mnemos-M	0.50 [0.50, 0.50]	0.00	1.00
model: qwen2.5	Mnemos-P	0.62 [0.57, 0.67]	0.74	0.97
model: qwen2.5	Mnemos-M	0.50 [0.50, 0.50]	0.74	0.97

TABLE 13
Recovering the keyed partition permutation from search traffic alone, with no induced writes, by matching the observed region co-occurrence graph against the one the public table predicts ($B = 64$, $k_{\mathrm{probe}} = 4$, 800 queries, null $1/B = 0.016$), over 20 independent stores. *FAQ* is the fast approximate QAP solver alone; *refined* adds pairwise-swap local search from each of its restarts and is the adversary reported, since a leakage measurement is only as strong as its attacker. Applies to the RW class only.

Probe mechanism	Exact, FAQ	Exact, refined	Neighbourhood
Genuine probes only ($s = 1$)	0.887 [0.823, 0.934]	0.963 [0.897, 1.000]	0.965
Randomised superset ($s = 2$, shipped)	0.393 [0.328, 0.460]	0.564 [0.469, 0.664]	1.000
Grouped probes (group size 8)	0.025 [0.016, 0.034]	0.025 [0.016, 0.034]	0.172

TABLE 14
What a sequence of file versions reveals about writes made *before* the compromise ($B = 64$, $w = 4$, null precision $1/B = 0.0156$). *Exposure* is how often the record's true region is inside the diff; *precision* is the chance of naming it by guessing within the diff.

Capture	Exposure	Precision [95% CI]	Candidate regions
every 1 flushes	1.000	0.208 [0.207, 0.208]	4.9
every 2 flushes	1.000	0.108 [0.108, 0.108]	9.3
every 4 flushes	1.000	0.058 [0.058, 0.058]	17.4
every 16 flushes	1.000	0.022 [0.022, 0.022]	46.2
every 64 flushes	1.000	0.016 [0.016, 0.016]	63.8
across a session, without re-encryption	1.000	0.0492 [0.0476, 0.0508]	20.4
across a session, with re-encryption	1.000	0.0156 [0.0156, 0.0156]	64.0

TABLE 15
The refusal channel ($B = 64$, $C = 32$, $w = 4$; padding alone rewrites a given region on 0.076 of flushes). The adversary induces writes destined for a region and watches whether that region is rewritten.

Refusal behaviour	AUC [95% CI]	Rate, full	Rate, open	Gap
Silent (writes nothing)	1.000 [1.000, 1.000]	0.070	1.000	+0.930
Padded (rewrites the region anyway)	0.500 [0.500, 0.500]	1.000	1.000	+0.000

TABLE 16
Capacity under a skewed write trace: when each design first refuses. The partitioned design begins refusing writes, and so leaves the non-overflowing regime of Lemma 2, long before the store is full; Theorem 1 still holds there, through padded refusal.

Design	Admitted	First refusal after	Utilisation
p	1845	488	23.8%
m	2048	2048	100.0%

TABLE 17
nDCG@10 against BEIR relevance judgments on the *full* corpora. The columns separate the two sources of loss: the gap from exhaustive to IVF at the same $k_{\mathrm{probe}} = 4$ is the cost of fixed probing, and the gap from exhaustive to Mnemos-M, which searches exactly, is the cost of int8 quantisation. *M (IVF)* is Mnemos-M with its in-memory partition index probing 8 partitions, the number of regions Mnemos-P reads at s=2. Refusals during indexing are counted, not swallowed.

		Plaintext float		int8 engines				
Domain	$\|D\|$	exhaustive	IVF@k	M (exact)	M (IVF)	P (s=2)	P grouped	Refused
scifact	5,183	0.738	0.657	0.737	0.703	0.658	0.435	0
nfcorpus	3,633	0.370	0.313	0.369	0.342	0.319	0.256	0
fiqa	57,638	0.384	0.327	0.384	0.358	0.329	0.224	0

TABLE 18
Fetching the top_k payloads, for the RW class. Path ORAM is a cost model with its parameters stated ($Z = 4$), not a measurement.

Method	n	Source	$p50$ (ms)	kB read/search	Leaks positions
resident	1000	measured	0.159	0.0	no
position	1000	measured	0.226	18.9	yes
read all	1000	measured	3.071	5691.1	no
path oram	1000	computed	—	1665.0	no
resident	10000	measured	1.751	0.0	no
position	10000	measured	1.787	18.9	yes
read all	10000	measured	30.477	56911.4	no
path oram	10000	computed	—	2119.0	no
resident	50000	measured	12.532	0.0	no
position	50000	measured	12.214	18.9	yes
read all	50000	measured	154.072	283830.3	no
path oram	50000	computed	—	2573.1	no

TABLE 19
Whether a write reaches storage depends on whether the content is already stored, read from each framework's shipped source (`pip download --no-deps`, 2026-09-27). *Yes* is recorded only where the mechanism was located at the cited file; anything not located is *none found* or *not verified*, never inferred from documentation. Where it is yes, the source satisfies the precondition of Proposition 6. Whether the oracle opens when the pipeline runs depends on the deciding model: run end to end it is open in mem0 with `qwen2.5` and closed in LangMem, which re-stores the memories it retrieves on every call (Table 4).

Framework	Version	Default store	Write depends on existing content	Evidence
mem0	2.2.1	Qdrant, local mode at /tmp/qdrant	**yes**	mem0/memory/main.py:925-955
CrewAI	1.15.22	LanceDB	**yes**	crewai/memory/unified_memory.py:93
LangMem	0.0.30	LangGraph BaseStore (backend chosen by the caller)	**yes**	langmem/knowledge/extraction.py:195-211
Graphiti (Zep)	0.30.2	graph database (caller-configured)	**yes**	graphiti_core/utils/maintenance/combined_extraction.py:171-174
LlamaIndex core	0.14.25	caller-chosen vector store	none found	llama_index/core/memory/memory_blocks/vector.py:29
Cognee	1.6.1	LanceDB (vectors), Kuzu (graph)	not verified	cognee/api/v1/config/config.py:215
Agno	3.0.11	caller-chosen database	none found	agno/db/base.py:580-636
Memobase	0.0.27	server-side (client package)	none found	–
Letta	0.33.2	server-side	not verified	–